LangGraph: A Complete Guide

Introduction:

LangGraph is a framework built on top of LangChain that helps developers create stateful, multi-step AI workflows using graph-based structures.

Key Concepts:

1. Nodes: Represent individual functions or steps in the workflow.

2. Edges: Define transitions between nodes.

3. State: Shared data that flows through the graph.

4. Conditional Routing: Allows dynamic decision-making between nodes.

Why Use LangGraph?

- Enables complex workflows like agents and RAG pipelines
- Supports looping and branching logic
- Maintains state across multiple steps

Basic Workflow:

1. Define state schema
2. Create nodes (functions)
3. Add edges (flow control)
4. Compile the graph
5. Execute the graph

Example Use Cases:

- Chatbots with memory
- Retrieval-Augmented Generation (RAG)
- Multi-agent systems

LangGraph + RAG Flow:

1. Load documents (PDF loader)
2. Split into chunks
3. Generate embeddings
4. Store in vector database
5. Retrieve relevant chunks

6. Generate response using LLM

Conclusion:

LangGraph is powerful for building production-grade AI systems with complex logic and workflows.